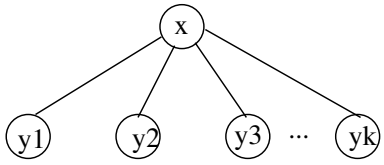
Problémam-megoldási stratégiák: Visszalépéses keresés, Keresés elágazás-korlátozással

Szerző: Horváth Gyula

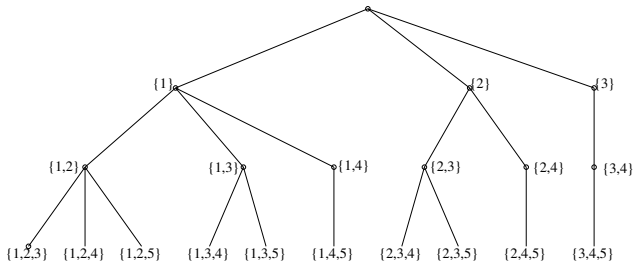
2025. november 18.

1. Megoldás keresése visszalépéssel (Backtracking)

A módszer olyan problémák megoldására alkalmazható, ahol a megoldás kifejezhető $X = \langle x_1, \ldots, x_k \rangle$ sorozat formájában. A lehetséges X megoldások kezdőszeletei , a **megoldáskezdemények** egy fát alkotnak, ezt nevezzük a probléma **megoldásterének**. Ellentétben a kimerítő kereséssel, nem csak a lehetséges megoldásokat vesszük sorba, hanem a megoldáskezdeményeket is. Ez lehetővé teszi, hogy ha egy megoldáskezdeményről kiderül, hogy semmilyen folytatása nem lehet megoldása a problémának, akkor az összes folytatását kizárjuk a keresésből.
A megoldáskezdemények gondolatilag alkotnak fát, csak azokat a fapontokat tároljuk megfelelő adatszerkezettel, amelyekre szükség van a keresés adott állapotában. Ha az x megoldáskezdeménynek $y_1, \ldots, y_k$ a közvetlen folytatásai, akkor a megoldástér fájában $y_1, \ldots$ és y_k gyermekei lesznek az x-et reprezentáló fapontnak. Például, ha a megoldandó probléma megoldása kifejezhető $(5, 3)$ kombinációként, akkor a megoldástér ábrázolható az alábbi fával, ahol

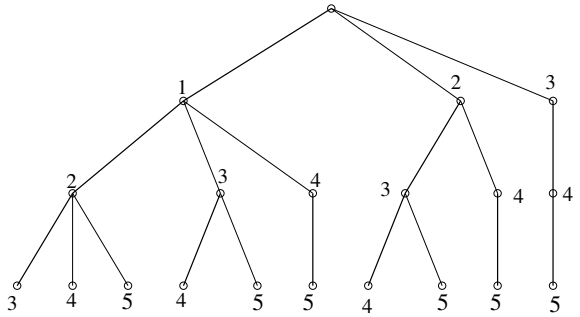


1. ábra



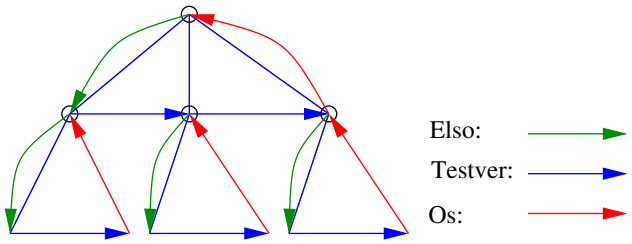
2. ábra. (5,3) kombinációk fája

Gyakran nem a kezdőszeleteket jelöljük a fa pontjaként, hanem a kezdőszeletet a gyökértől az adott pontig vezető úton



3. ábra

lévő adatok sorozata adja. Et jobban kifejezi azt, hogy a megoldáskezdeményeket lépésről lépésre építjük a bejárás során. Tehát minden fabejáró algoritmus alkalmazható megoldás keresésére és a bejárás során alkalmazhatjuk a kizárást. Ahhoz, hogy a megoldástere mint fát bejárjuk rekurzív algoritmussal, elegendő két művelet. Ha az `x` megoldáskezdemény közvetlen lehetséges folytatásai $y_1, \ldots, y_k$, akkor kell egy művelet, amely `x`-ből előállítja az első folytatást, azaz y_1-et, és kell egy olyan művelet, amely y_i-ből előállítja y_{i+1}-et. Persze azt is jeleznie kell, ha nincs következő testvért. Ha iteratívval akarjuk bejárni a megoldásteret, akkor kell még egy olyan művelet, amely y_k-ból előállítja `x`-et.



4. ábra. Fa bejáráshoz alkalmazandő műveletek

1.1. Alapműveletek fák bejárásához

`bool Elso(FaPont& X)`
Ha létezik az `X` fapontnak első gyereke, akkor `X` értéke az első gyerek lesz és igaz értéket ad vissza, egyébként `X` értéke nem változik és hamis értéket ad vissza.
`bool Testver(FaPont& X)`
Ha létezik az `X` fapontnak következő testvére, akkor `X` értéke a következő testvér lesz és igaz értéket ad vissza, egyébként `X` értéke nem változik és hamis értéket ad vissza.
`bool Os(FaPont& X)`
Ha létezik az `X` fapontnak közvetlen őse, akkor `X` értéke a közvetlen ős lesz és igaz értéket ad vissza, egyébként `X` értéke nem változik és hamis értéket ad vissza.

Megjegyezzük, hogy az `Os` műveletet csak olyan fapontra kell megvalósítani, amelynek nincs testvére.

1.2. Fa rekurzív bejárása mentéssel

Mivel az **Elso(X)** művelet megváltoztatja `X` értékét, ezért a művelet előtt el kell menteni

```
1 void rFaBejar(FaPont X){
2   //rekurzív fabejárás
3   //az X fapont feldolgozása
4   X.KiIr();
5   if(!X.Elso()) return;
6   do{
7     rFaBejar(X);
8   }while(X.Testver());
9 }
```

1.3. Fa rekurzív bejárása visszaállítással

Ha megadjuk az Elso() műveket iElso(), illetve a Testver() művelet iTestver() inverzét, akkor a rekurzív bejárást ezekkel is megvalósíthatjuk. Az inverz műveltek visszaállítják a fapont művelet előtti értékét.

```
1  void iFaBejar(FaPont& X){
2    //az X fapont feldolgozása
3    if(Elso(X)){
4      iFaBejar(X);
5      iElso(X);
6    }
7    if(Testver(X)){
8      iFaBejar(X);
9      iTestver(X);
10   }
11 }
```

1.4. Fa iteratív bejárása

```
1  void FaBejar(FaPont& X){
2    //iteratív fa bejárás
3    bool tovabb=true;
4    bool elsore=true;
5    while(tovabb){
6      if(elsore){
7        //az X fapont feldolgozása
8        elsore=Elso(X);
9      }else{
10       if(Testver(X))
11         elsore=true;
12       else//nincs testvére, visszalépés
13         tovabb=Os(X);
14     }
15   }
16 }
```

A megoldás keresését meg tudjuk fogalmazni olyan általános formában, hogy az algoritmus érdemi része, azaz a megoldástér bejárása csak néhány problémaspecifikus műveletet alkalmaz.
Ezt módszert "application framework" módszernek is nevezik. Adott problémára nem kell újraírni az érdemi részt, csak a problémaspecifikus műveletek megvalósítását kell megadni.
A fabejáró műveletek mellett még két probléma-specifikus művelet kell; a **Lehet()** kizáró művelet, és **Megoldas**(), amely teszteli, hogy az adott megoldáskezdemény megoldása-e a problémának.

bool X.Lehet();
Akkor és csak akkor adjon hamis értéket, ha biztosan nincs megoldás a megoldástér `X`-gyökerű részfájában. Ha igaz értéket ad, abból nem következik, hogy létezik is megoldás az `X`-gyökerű részfában.`X` értéke nem változhat.
Tehát hamisat ad, ha az `X` fapont által reprezentált megoldáskezdeménynek nincs olyan folytatása, amely megoldása a problémának.
bool X.Megoldas();
Akkor és csak akkor adjon igaz értéket, ha `X` fapont által reprezentált megoldáskezdemény megoldása a problémának. `X` értéke nem változhat.
A továbbiakban `FaPont` azonosítót használunk a megoldástér pontjainak típusára. Az `FaPont` adatszerkezet, amely tartalmazza a megoldáskezdeményt és olyan kiegészítő adatokat, amelyek segítik a műveletek (hatékony megvalósítását.

```
class FaPont{//a megoldástér pontja
  //Probléma-specifikus adatoktagok
  public:
  bool Elso();
  bool Testver();
  bool Megoldas();
  bool Lehet();
//   bool Os(); //nemrekurzív keresés esetén
  FaPont(){//üres megoldáskezdeményt létesítő konstruktor
  }
  void KiIr();
};
```

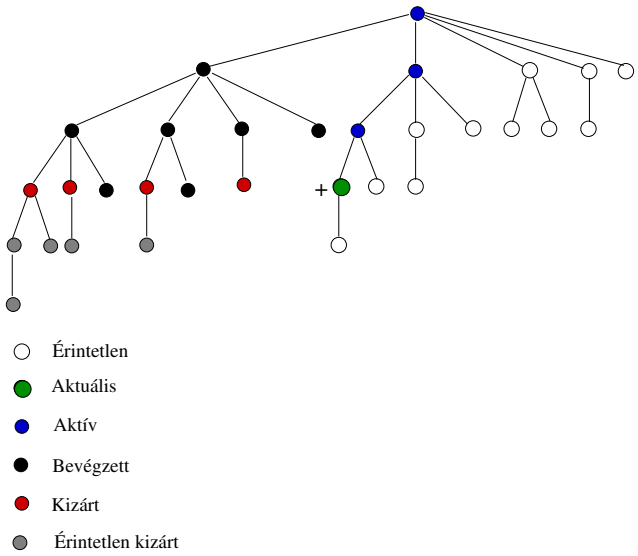
1.5. Rekurzív visszalépéses keresés algoritmusa

```
FaPont M=FaPont(); //egy megoldás
bool van=false;
void rKeres(FaPont X){
  if(van) return;//ha egy megoldást keresünk
  if(!X.Lehet()) return;
  if(X.Megoldas()){
    M=X; X.KiIr();
     van=true;
     return;//ha egy megoldást keresünk
  }//megoldást találtunk

  if(!X.Elso()) return;
  do{
     rKeres(X);
  }while(X.Testver());
}
```

1.6. Iteratív visszalépéses keresés algoritmusa

```
bool Keres(FaPont &X){
  //Nemrekurzív visszalépéses keresés
  bool tovabb=X.Lehet();
  bool elsore=true;
  while(tovabb){
     if(elsore){
        if(X.Megoldas()){return true;}//egy megoldás
        elsore=X.Lehet() && X.Elso();//kizárás ha !Lehet
     }else{
        if(X.Testver())
           elsore=true;
        else//visszalépés
           tovabb=X.Os();
     }
  }
  return false;
}
```

1.7. A megoldástér pontjainak osztályozása



5. ábra. A megoldástér pontjainak osztályozása visszalépéses keresésnél

Érintetlen: az olyan pontja a megoldástérnek, amelyet a keresés során még nem érintettünk.

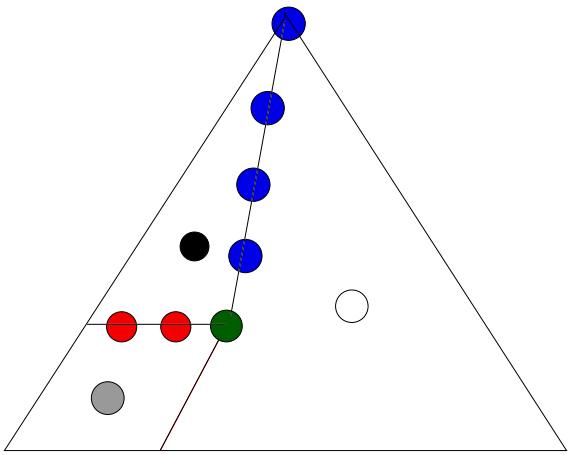
Aktuális: az a pont, amelyet éppen vizsgálunk.

Aktív az olyan pont, amelyet már érintettünk a keresés során, de még nem bevégzett. Pontosan azok az aktív pontok, amelyek az aktuális pont ősei a fában. A keresés során az aktív pontokba még visszatérünk.

Kizárt: olyan megoldáskezdemény, amelynek egyetlen folytatása (leszármazottja a fában) sem lehet megoldás.

Bevégzett: minden fia vagy kizárt, vagy bevégzett.

Érintetlen-kizárt: leszármazottja valamely kizárt pontnak. Tehát a megoldástér ezen pontjait nem érinti a keresés.

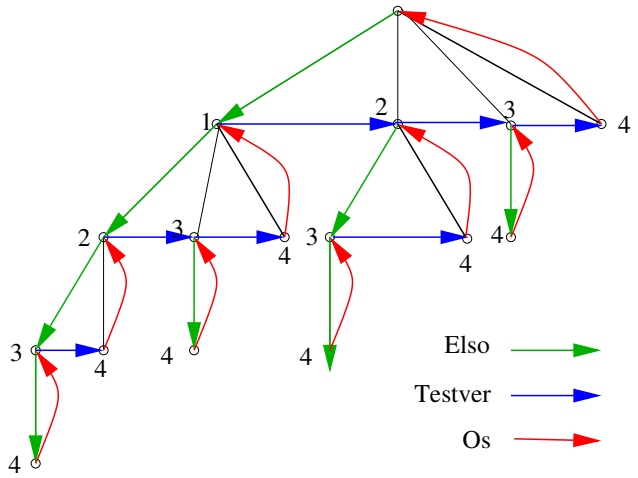


6. ábra. A megoldástér sematikus képe visszalépéses keresésnél.

A továbbiakban először megmutatjuk, hogyan lehet megvalósítani az egyes kombinatorikus struktúrák (permutációk, részhalmazok, kombinációk, variációk) esetén a fabejáró műveleteket.

1.8. Fabejáró műveletek `részhalmaz` megoldás esetén

Az `1, ..., n` alaphalmaz részhalmazai reprezentálhatók az elemek növekvő felsorolásával, vagy bitvektorral.

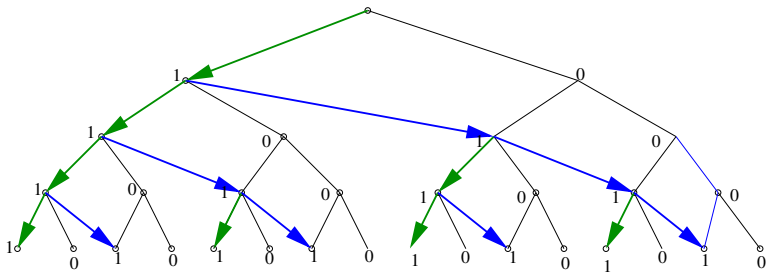


7. ábra. A megoldástér fája `n = 4` esetén

```
class FaPont{//a megoldástér pontja
  //Probléma-specifikus adattagok
  int n;
  vector<int> P;//egy részhalmaz
  public:
  bool Elso();
  bool Testver();
  bool Os();
  FaPont(int xn){ n=xn; }
  void KiIr(){
    for(int x:P) cout<<x<<" "; cout<<endl;
  }
};

bool FaPont::Elso(){
  int u=P.size()==0 ? 0: P.back();
  if(u==n) return false;
  P.push_back(u+1);
  return true;
}
```

```
bool FaPont::Testver(){
  if(P.size()==0 || P.back()==n) return false;
  P[P.size()-1]++;
  return true;
}

bool FaPont::Os(){
  if(P.size()==0) return false;
  P.pop_back();
  return true;
}
```

1.8.1. Bitvektoros reprezentáció



8. ábra. Részhalmazok bitvektoros reprezentációja $n = 5$ esetén

Minden részhalmazt olyan bitsorozat reprezentál, amelynek az utolsó tagja igaz értékű. Az Elso(R) és a Testver(R) műveletekkel generálhatjuk az összes részhalmazt.

```
class FaPont{//a megoldástér pontja
  //Probléma-specifikus adatoktagok
  int n;//az alaphalmaz: {1,...,n}
  vector<int> P;//egy reszhalmaz
  public:
  bool Elso();
  bool Testver();
  bool Os();
  FaPont(int xn){ n=xn;}
  void KiIr(){
    for(int i=0;i<P.size();i++)
    if(P[i]) {cout<<i+1<<",";} cout<<endl;
  }
};

bool FaPont::Elso(){
  if(P.size()==n) return false;
  P.push_back(1);
  return true;
}

bool FaPont::Testver(){
  if(P.size()==n) return false;
  if(P.size()==0 || P[P.size()-1]==0) return false;
  P[P.size()-1]=0;
  P.push_back(1);
  return true;
}

```

```
30 bool FaPont::Os(){
31   if(P.size()==0) return false;
32   P.pop_back();
33   return true;
34 }
35 void FaBejar(FaPont& F){
36   bool tovabb=true;
37   bool elsore=true;
38   while(tovabb){
39     if(elsore){
40       //a fapont feldolgozása
41       F.KiIr();
42       elsore=F.Elso();
43     }else{
44       if(F.Testver())
45       elsore=true;
46       else//nincs testvére, visszalépés
47       tovabb=F.Os();
48     }
49   }
50 }
51 void rFaBejar(FaPont X){
52   //rekurzív fabejárás
53   //az X fapont feldolgozása
54   X.KiIr();
55   if(!X.Elso()) return;
56   do{
57     rFaBejar(X);
58   }while(X.Testver());
59 }
```

2. A Pénzváltás feladat megoldása

Minden fapontban tároljuk kiegészítő adatként a megoldáskezdeményben lévő pénzek összegét a `resz` adattagban. Előzetesen számítsuk ki a `Maradt[i]` = $\sum_{j=i}^{n}$ `Penz[j]` szuffix összegeket. Ekkor korlátozó feltételként megadhatjuk azt, hogy `resz` + `Maradt[u + 1]` $\geq$ `K`, ahol `u` a megoldáskezdemény utolsó tagja (a legnagyobb pénz index).

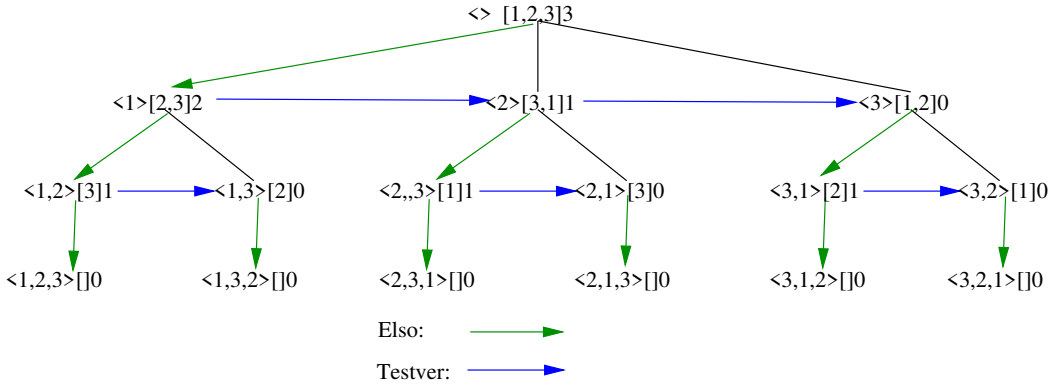
```
1   //A Pénzváltás feladat megoldása
2   #include <bits/stdc++.h>
3   using namespace std;
4   const int maxN=21;
5   int Penz[maxN];   //a pénzek
6   int Maradt[maxN];//szuffix összegek
7   int K;            //a kifizetendő
8   class FaPont{//a megoldástér pontja
9     //Globális: Penz[], K
10    //Probléma-specifikus adattagok
11    int n;
12    int resz=0;
13    vector<int> P;//egy részhalmaz
14    public:
15    bool Elso();
16    bool Testver();
17    bool Lehet();
18    bool Megoldas();
19
20    FaPont(int xn){ n=xn;}
21    void KiIr(){
22      for(int x:P) cout<<x<<" "; cout<<endl;
23    }
24    FaPont(){};
25  };
26
27  bool FaPont::Elso(){
28    int u=P.size()==0 ? 0: P.back();
29    if(u==n) return false;
30    P.push_back(u+1);
31    resz+=Penz[u+1];
32    return true;
33  }
34
```

```
35    bool FaPont::Testver(){
36      if(P.size()==0 || P.back()==n) return false;
37      resz-=Penz[P.back()];
38      P[P.size()-1]++;
39      resz+=Penz[P.back()];
40      return true;
41    }
42    bool FaPont::Megoldas(){
43      return resz==K;
44    }
45    bool FaPont::Lehet(){
46      int u=(P.size()==0) ? 1: P.back()+1;
47      if(u>n) return false;
48      return resz<=K && resz+Maradt[u]>=K;
49    }
50  FaPont M=FaPont();//egy megoldás
51  bool van=false;
52  void rKeres(FaPont X){
53    if(van) return;//ha egy megoldást keresünk
54    if(!X.Lehet()) return;
55    if(X.Megoldas()){
56      M=X;
57      van=true; //return;
58    }//megoldást találtunk
59
60    if(!X.Elso()) return;
61    do{
62      rKeres(X);
63    }while(X.Testver());
64  }
```

```
int main(){
  int N;
  cin>>K>>N;
  for(int i=1;i<=N;i++){
    cin>>Penz[i];
    Maradt[i]=0;
  }
  Maradt[N]=Penz[N];
  for(int i=N-1;i>=1;i--)
  Maradt[i]=Maradt[i+1]+Penz[i];
  FaPont F=FaPont(N);

  rKeres(F);
 M.KiIr();
}
```

2.1. Fabejáró műveletek `permutáció` megoldás esetén

2.1.1. Műveletek rekurzív bejáráshoz



9. ábra. A megoldástér fája $n = 3$ esetén

Négy adatot tárolunk a megoldástér pontjában: n a permutáció mérete, P =<> a permutáció kezdőszelete, $Q = []$ a P komplementere, tehát azon számok, amelyek még választhatók a permutáció következő tagjaként. A negyedik adattag az l, ami azt jelenti, hogy a Q sorozatból még az első l tag választható, amivel folytatható a P permutáció kezdőszelet. A Testver() művelettel mindig Q első tagját választjuk, P utolsó tagja pedig Q végére kerül. Ezért Q típusa kétvégű sor (deque) lesz. Ha $X =< p_1, \ldots, p_{k-1} > [p_k, \ldots, p_n]l$, akkor Elso(X) hatására
$X =< p_1, \ldots, p_{k-1}, p_k > [p_{k+1}, \ldots, p_n]n - k$ lesz, a Testver(X) hatására pedig
$X =< p_1, \ldots, p_{k-2}, p_k > [p_{k+1}, \ldots, p_n, p_{k-1}]l - 1$.

```
1  class FaPont{//a megoldástér pontja
2    int n;//permutáció mérete
3    int l;//léptetések száma
4    vector<int> P;//egy permutáció kezdőszelete
5    deque<int> Q; //P komplementere
6    public:
7    bool Elso();
8    bool Testver();
9    FaPont(int xn){
10     n=xn;l=n;
11     for(int x=1;x<=n;x++) Q.push_back(x);
12   }
13   void KiIr(){for(int x:P) cout<<x<<" "; cout<<endl;
14   }
15 };
16 bool FaPont::Elso(){
17   if(Q.size()==0) return false;
18   int q=Q.front(); Q.pop_front();
19   P.push_back(q);
20   l=Q.size();
21   return true;
22 }
```

```
23 bool FaPont::Testver(){
24   if(l==0) return false;
25   int q=Q.front(); Q.pop_front();
26   int p=P.back(); P.pop_back();
27   P.push_back(q);
28   Q.push_back(p);
29   l--;
30   return true;
31 }
```

2.1.2. Műveletek iteratív bejáráshoz

Adott n-re az összes permutáció és azoknak a kezdőszeleteinek előállítása, azaz a megoldástér iteratív bejárásához szükséges Os() művelet nem oldható meg (hatékonyan=konstans időben) azokkal az adatokkal, amelyeket a rekurzív esetben alkalmaztunk. Ugyanis, nem tudjuk, hogy az ős pontban mi volt az `l` értéke, pedig azt vissza kell állítani. Tehát az `l` skalár érték helyett egy `L` veremben tároljuk az adott faponthoz tartozó `l` értéket. Az ős pont `P` és `Q` értéke visszaállítható, úgy, hogy `P` utolsó tagját átrakjuk `Q` végére, mert a Testver() művelettel cirkuláris léptetést alkalmaztunk.

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adattagok
3    int n;//permutáció mérete
4    vector<int> P;//megoldáskezdemény
5    deque<int> Q; //P komplementere
6    stack<int> L; //léptetések szintenként
7    public:
8    bool Elso();
9    bool Testver();
10   bool Os();
11   FaPont(int xn){
12     n=xn;
13     for(int x=1;x<=n;x++) Q.push_back(x);
14   }
15   void KiIr(){
16     for(int x:P) cout<<x<<" "; cout<<endl;
17   }
18 };
19 bool FaPont::Elso(){
20   if(Q.size()==0) return false;
21   int q=Q.front(); Q.pop_front();
22   P.push_back(q);
23   L.push(Q.size());
24   return true;
25 }
```

```
bool FaPont::Testver(){
  int l=L.top(); L.pop();
  if(l==0){return false;}//nincs már testvér
  int q=Q.front(); Q.pop_front();
  int p=P.back(); P.pop_back();
  P.push_back(q);
  Q.push_back(p);
  l--; L.push(l);
  return true;
}
bool FaPont::Os(){
  if(P.size()==0) return false;
  int p=P.back(); P.pop_back();
  Q.push_back(p);
  return true;
}
```

3. Az Ültetés feladat megoldása

Jelölje `F` feltétel párokat, tehát az `i`-edik tag azt kéri, hogy mellette vagy `F[i].first`, vagy `F[i].second` üljön valamelyik oldalán. A szomszédság cirkulárisan értendő. Tehát az `1, ..., n` halmaznak olyan permutációját keressük, amely minden tag kérését teljesíti, ha van ilyen.
Korlátozó feltételként megadhatjuk azt, hogy a megoldáskezdemény utolsó (nem `n` értékű) tagjának nem felel meg az előtte ülő, akkor következő tagként csak olyan választható, amely egy alkalmas szomszédja lehet az utolsónak.

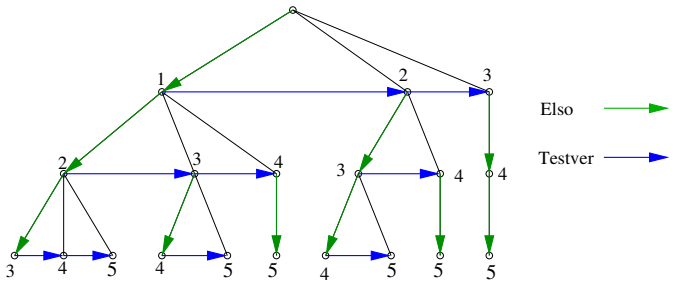
```
1   //Az Ültetés feladat megoldása
2   #include <bits/stdc++.h>
3   using namespace std;
4   vector<pair<int,int>> F;
5
6   class FaPont{//a megoldástér pontja
7     //Globális: F
8     //Probléma-specifikus adattagok
9     int n;//permutáció mérete
10    int l;//léptetések száma
11    vector<int> P;//megoldáskezdemény
12    deque<int> Q; //P komplementere
13    public:
14    bool Elso();
15    bool Testver();
16    bool Lehet();
17    bool Megoldas();
18    FaPont(int xn){
19      n=xn;l=n;
20      for(int x=1;x<=n;x++) Q.push_back(x);
21    }
22    FaPont(){};
23    void KiIr(){
24      for(int x:P) cout<<x<<" "; cout<<endl;
25    }
26  };
27
28  bool FaPont::Elso(){
29    if(Q.size()==0) return false;
30    int q=Q.front(); Q.pop_front();
31    P.push_back(q);
32    l=Q.size();
33    return true;
34  }
```

```
bool FaPont::Testver(){
  if(l==0) return false;
  int q=Q.front(); Q.pop_front();
  int p=P.back(); P.pop_back();
  P.push_back(q);
  Q.push_back(p);
  l--;
  return true;
}
bool FaPont::Lehet(){
  if(P.size()==n) return false;
  if(P.size()<=1) return true;
  int u=P.back(); int u1=P[P.size()-2];
  int e=Q.front();
  return  F[u].first==u1 || F[u].second==u1 ||
  F[u].first==e || F[u].second==e ;
}

bool FaPont::Megoldas(){
  if(P.size()<n) return false;
  int kovet,eloz;
  bool van=true;
  for(int i=0;i<n;i++){
    kovet=P[(i+1)%n]; eloz= i>0 ? P[i-1]: P[n-1];
    if( (F[P[i]].first!=kovet && F[P[i]].second!=kovet) &&
    (F[P[i]].first!=eloz && F[P[i]].second!=eloz) ){
      van=false; break;
    }
  }
  return van;
}
```

```
27    FaPont M=FaPont();//egy megoldás
28    bool van=false;
29    void rKeres(FaPont X){
30      if(van) return;//ha egy megoldást keresünk
31      if(!X.Lehet()) return;
32      if(X.Megoldas()){
33        M=X; X.KiIr();
34        van=true; //return;
35      }//megoldást találtunk
36
37      if(!X.Elso()) return;
38      do{
39        rKeres(X);
40      }while(X.Testver());
41    }
42
43    int main(){
44      int n,x,y;
45      F.push_back({0,0});
46      cin>>n;
47      for(int i=1;i<=n;i++){
48        cin>>x>>y;
49        F.push_back({x,y});
50      }
51      FaPont F=FaPont(n);
52      rKeres(F);
53      //cout<<"mego\n";
54      if(van) M.KiIr();
55    }
```

3.1. Fabejáró műveletek `kombináció` megoldás esetén

Az `(n, k)` kombinációk pontosan `k` elemű részhalmazok, ezért az összes kombináció, illetve a kezdőszeleteik előállítása hasonló a részhalmazokéhoz.



10. ábra. `(5, 3)` kombinációk fája.

3.1.1. Műveletek rekurzív bejáráshoz

```
class FaPont{//a megoldástér pontja
  //Probléma-specifikus adatoktagok
  int n;//
  int k;//(n,k) kombinációk
  vector<int> P;//egy reszhalmaz
  public:
  bool Elso();
  bool Testver();
  FaPont(int xn, int xk){
    n=xn; k=xk;
  }
  void KiIr(){
    for(int x:P) cout<<x<<" "; cout<<endl;
  }
};
```

```
bool FaPont::Elso(){
  int l=P.size();
  if(l==k) return false;
  int p=(l==0) ? 0 : P.back();
  if(p+k-l>n) return false;
  P.push_back(p+1);
  return true;
}
bool FaPont::Testver(){
  int l=P.size();
  if(l==0) return false;
  int p= P.back();
  if(p+k-l>=n) return false;
  P[l-1]++;
  return true;
}
```

3.1.2. Műveletek iteratív bejáráshoz

Az Elso() és a Testver() művelet ugyanaz, mint a rekurzív esetben,

```
bool FaPont::Os(){
  int l=P.size();
  if(l==0) return false;
  P.pop_back();
  return true;
}
```

4. A Hozzárendelés feladat megoldása

Jelölje `VM[v]` azon munkák halmazát, amelyeket a `v` alvállalkozó el tud végezni.
A feladat megoldása megadható egy olyan $f : \{1, \ldots, M\} \to \{1, \ldots, V\}$ függvénnyel, amelyre teljesül az alábbi két feltétel:

$$f(i) \neq f(j) \text{ minden } 1 \leq i < j \leq M \text{ esetén}$$

$$m \in VM[f(m)] \text{ minden } 1 \leq m \leq M \text{ esetén}$$

Tehát a megoldás megadható az `(f(1), ..., f(M))` sorozattal, ami nem más, mint `V`-nek `M`-ad osztályú variációja.
Tehát minden lehetséges megoldása a feladatnak megadható egy-egy variációval.
Mivel `V` értéke nem nagy. ezért minden `VM[v]` halmazt reprezentálhatunk bitvektorral.

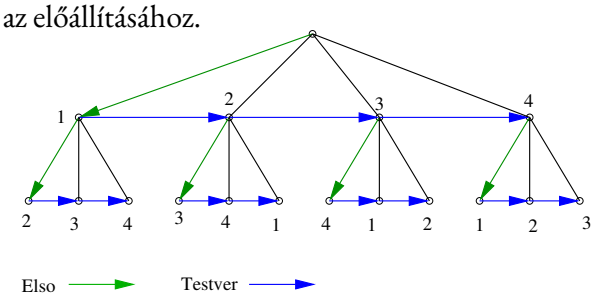
```
1  #include <bits/stdc++.h>
2  using namespace std;
3  const int maxN=21;
4
5  bool VM[maxN][maxN];
6  //a v tudja az m munkát, ha V[v][m] igaz
7  class FaPont{//a megoldástér pontja
8    //Globális: VM[][]
9    //Probléma-specifikus adatoktagok
10   int n;//
11   int k;//(n,k) variácok fája
12   int l;//testver léptetések száma
13   vector<int> P;//egy reszhalmaz
14   deque<int> Q; //P komplementere
15   public:
16   bool Elso();
17   bool Testver();
18   bool Lehet();
19   bool Megoldas();
20   FaPont(int xn, int xk){
21     n=xn; k=xk; l=n;
22     for(int x=1;x<=n;x++) Q.push_back(x);
23   }
24   FaPont(){};
25   void KiIr(){
26     for(int x:P) cout<<x<<" "; cout<<endl;
27   }
28 };
```

```
bool FaPont::Elso(){
  if(P.size()==k) return false;
  if(Q.size()==0) return false;
  int q=Q.front(); Q.pop_front();
  P.push_back(q);
  l=Q.size();
  return true;
}
bool FaPont::Testver(){
  if(l==0) return false;
  int q=Q.front(); Q.pop_front();
  int p=P.back(); P.pop_back();
  P.push_back(q);
  Q.push_back(p);
  l--;
  return true;
}

bool FaPont::Lehet(){
  //  return true;
  if(P.size()==0) return true;
  return VM[P.size()][P.back()];
}

bool FaPont::Megoldas(){
  return (P.size()==k);
}
```

```
100 FaPont M=FaPont(); //egy megoldás
101 bool van=false;
102 void rKeres(FaPont X){
103   if(van) return;//ha egy megoldást keresünk
104   if(!X.Lehet()) return;
105
106   if(X.Megoldas()){
107     M=X; X.KiIr();
108      van=true; return;
109   }//megoldást találtunk
110   if(!X.Elso()) return;
111   do{
112      rKeres(X);
113   }while(X.Testver());
114 }
115 int main(){
116   int m,v,x;
117   string s;
118   cin>>m>>v;
119   getline(cin, s);
120   for(int i=1;i<=v;i++)
121   for(int j=1;j<=m; j++) VM[j][i]=false;
122   for(int i=1;i<=v;i++){
123      while(cin.peek()!='\n'){
124         cin>>x;
125        VM[x][i]=true;
126      }
127      getline(cin, s);
128   }
129   FaPont VM=FaPont(v, m);
130   rKeres(VM);
131   if(van) M.KiIr();
132 }
```

4.1. Fabejáró műveletek `variáció` megoldás esetén

Mivel a `(n, k)` variációk az `n` méretű permutációk `k` méretű különböző kezdőszeletei, ezért az előállításuk hasonló a permutációk és kezdőszeleteinek az előállításához.



11. ábra. `(4, 2)` vatiációk fája

4.1.1. Műveletek rekurzív bejáráshoz

```
1  class FaPont{//a megoldástér pontja
2    //Probléma-specifikus adatoktagok
3    int n;//
4    int k;//(n,k) variácok fája
5    int l;//testver léptetések száma
6    vector<int> P;//egy reszhalmaz
7    deque<int> Q; //P komplementere
8    public:
9    bool Elso();
10   bool Testver();
11   FaPont(int xn, int xk){
12     n=xn; k=xk; l=n;
13     for(int x=1;x<=n;x++) Q.push_back(x);
14   }
15   void KiIr(){
16     for(int x:P) cout<<x<<" "; cout<<endl;
17   }
18 };
19
20 bool FaPont::Elso(){
21   if(P.size()==k) return false;
22   if(Q.size()==0) return false;
23   int q=Q.front(); Q.pop_front();
24   if(q==0) return false;
25   P.push_back(q);
26   l=Q.size();
27   return true;
28 }
```

```
29 bool FaPont::Testver(){
30   if(l==0) return false;
31   int q=Q.front(); Q.pop_front();
32   int p=P.back(); P.pop_back();
33   P.push_back(q);
34   Q.push_back(p);
35   l--;
36   return true;
37 }
```

4.1.2. Műveletek iteratív bejáráshoz

```
1 class FaPont{//a megoldástér pontja
2   //Probléma-specifikus adatoktagok
3   int n;//
4   int k;//(n,k) variácok fája
5   vector<int> P;//egy reszhalmaz
6   deque<int> Q; //P komplementere
7   stack<int> L; //léptetesek szintenként
8   public:
9   bool Elso();
10  bool Testver();
11  bool Os();
12  FaPont(int xn, int xk){
13    n=xn; k=xk;
14    for(int x=1;x<=n;x++) Q.push_back(x);
15  }
16  void KiIr(){
17    for(int x:P) cout<<x<<" "; cout<<endl;
18  }
19 };
20
21 bool FaPont::Elso(){
22   if(Q.size()==0) return false;
23   if(P.size()==k) return false;
24   int q=Q.front(); Q.pop_front();
25   P.push_back(q);
26   L.push(Q.size());
27   return true;
28 }
```

```
bool FaPont::Testver(){
  int l=L.top(); L.pop();
  if(l==0){return false;}
  int q=Q.front(); Q.pop_front();
  int p=P.back(); P.pop_back();
  P.push_back(q);
  Q.push_back(p);
  l--; L.push(l);
  return true;
}
bool FaPont::Os(){
  if(P.size()==0) return false;
  int p=P.back(); P.pop_back();
  Q.push_back(p);
  return true;
}
```

5. A Koalíciók feladat megoldása

Minden `k` tagú koalíció egy `(p, k)` kombinációval adható meg, ahol `p` a parlamenti pártok száma. Az összes `(p, k)` kombinációt sorra kell venni, és ha többségi, akkor növeljük minden tagjának a megoldás számlálóját.

```
//A Koalíciók feladat megoldásda
#include <bits/stdc++.h>
using namespace std;
vector<int> M;//mandátumok szána
int Ossz=0;
vector<int> K;
//a p párt K[p] koalícióban lehet tag

class FaPont{//a megoldástér pontja
  //Globális: Ossz, K
  //Probléma-specifikus adatoktagok
  int n,k;
  public:
  vector<int> P;//egy reszhalmaz
  bool Elso();
  bool Testver();
  bool Lehet();
  bool Megoldas();

  FaPont(int xn, int xk){
    n=xn; k=xk;
  }
  void KiIr(){
    for(int x:P) cout<<x<<" "; cout<<endl;
  }
  FaPont(){};
};

bool FaPont::Elso(){
  int l=P.size();
  if(l==k) return false;
  int p=(l==0) ? 0 : P.back();
  if(p+k-l>n) return false;
  P.push_back(p+1);
  return true;
}
```

```
37 bool FaPont::Testver(){
38   int l=P.size();
39   if(l==0) return false;
40   int p= P.back();
41   if(p+k-l>=n) return false;
42   P[l-1]++;
43   return true;
44 }
45 bool FaPont::Lehet(){
46   return P.size()<=k;
47 }
48
49 bool FaPont::Megoldas(){
50   if(P.size()!=k) return false;
51   int om=0;
52   for(int x:P) om+=M[x];
53   return 2*om>Ossz;
54 }
55
56 void rKeres(FaPont X){
57   if(!X.Lehet()) return;
58   if(X.Megoldas()){
59     for(int x:X.P) K[x]++;
60   }//megoldást találtunk
61
62   if(!X.Elso()) return;
63   do{
64     rKeres(X);
65   }while(X.Testver());
66 }
```

```
int main(){
  int p,k,x;
  cin>>p>>k;
  K.push_back(0);
  M.push_back(0);
  for(int i=1;i<=p;i++){
     cin>>x;
    M.push_back(x);
    K.push_back(0);
     Ossz+=x;
  }
  FaPont F=FaPont(p, k);
  rKeres(F);
  for(int i=1;i<=p;i++) cout<<K[i]<<" "; cout<<endl;
}
```

5.0.1. Visszalépéses keresési algoritmusok futási ideje

Legrosszabb esetben a keresés a megoldástér-fa minden pontját bejárja, aminek mérete általában exponenciális a megoldásvektor méretének függvényében.

5.0.2. Visszalépéses keresési algoritmusok tárigénye

Rekurzív megoldás esetén a rekurzió mélysége határozza meg a felhasznált memóriát.
Iteratív megvalósítás esetén a megoldástérnek csak egyetlen pontját kell tárolni.
A visszalépéses keresés alkalmazható optimális megoldás előállítására is.
Legyen `Cf(X)` a célfüggvény, tehát olyan `X`-et keresünk, amelyre: *Megoldas* `(X)` és `Cf(X)` minimális.
Az összes megoldás keresése során ha jobb megoldást találunk, feljegyezzük.

6. Megoldás keresés elágazás-korlátozással (Branch and bound)

Úgy tekintjük, hogy a megoldásteret alkotó megoldáskezdemények fát alkotnak, ezért minden fabejáró módszer (algoritmus) alkalmazható megoldás keresésére. A visszalépéses keresés a mélységi bejárás stratégiáját alkalmazza, aminek az a lényege, hogy a keresés során mindig egy aktív pontot egy lehetséges folytatásával folytatja a keresés, és visszatérést hajt végre, ha nem lehet folytatni.
Nyilvánvalóan lehet alkalmazni olyan bejárási stratégiát, amely az aktív pont minden lehetséges közvetlen folytatását kigenerálja, ezeket tárolja alkalmas tárolóban és innen választ következő aktív pontot. Ezen stratégia algoritmusa megfogalmazható általánosan, ahol az aktív pontokat egy adagoló tárolja.
Adagolón olyan adattípust értünk, amelynek értékhalmaza adott `E` elemtípusbeli véges halmazok és az alábbi három művelete van:
void Betesz(E X)
Az `X` adatelemet beteszi a halmazba.
E Kivesz() A halmaz egy tetszőleges elemét adja vissza, amelyet kivesz a halmazból.
bool Ures()
Akkor és csak akkor ad igaz értéket, ha a halmaz üres.

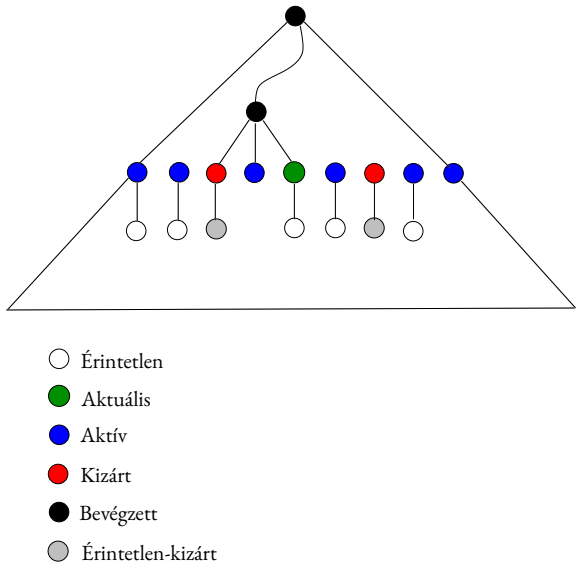
6.1. Megoldás keresése adagolós bejárással

```
1 class FaPont{
2   //probléma-specifikus adatoktagok
3   public:
4   bool Elso(){};
5   bool Testver(){};
6   bool Lehet(){};
7   bool Megoldas(){};
8   FaPont(){};
9 };
```

```
10 class Adagolo{
11   public:
12   void Betesz(FaPont X){};
13   FaPont Kivesz(){};
14   bool Ures(){};
15 };
16
17 FaPont AKeres(FaPont X){
18   //Egy megoldás keresése az X gyökerű
19   //megoldástér-fában
20   Adagolo A;
21   FaPont M;//egy megoldás
22   if (!X.Lehet()) return M;
23   A.Betesz(X);
24   while (!A.Ures()){
25     X=A.Kivesz();
26     //X közvetlen leszármazottainak (gyerekeinek) aktiválása
27     if(!X.Elso()) continue;//X-nek nincs folytatása
28     do{//X minden testvérére
29       if (!X.Lehet()) continue;//kizárás
30       if (X.Megoldas()){//megoldást találtunk
31         M=X;
32         break;//ha egy megoldást kerestünk
33       }
34       A.Betesz(X);
35     }while(X.Testver());
36   }//while
37   return M;
38 }
```

Az adagoló lehet (többek között) verem, sor vagy prioritási sor.
Sőt, az aktuális pontot akár véletlenszerűen is választhatjuk az aktívak közül. Tágabb értelemben minden olyan megoldási módszert, amely adagolóval történő keresést alkalmaz, elágazás-korlátozás elnevezéssel illetnek (Brach and Bound). Szűkebb értelemben csak azokat a módszereket, amelyeknél az adagoló prioritási sor. A módszert leginkább optimalizációs feladatok megoldására alkalmazzák, amikor bemeneti adatként adott egy `Cf` célfüggvény, és olyan megoldást keresünk, amelyre `Cf` értéke minimális/maximális.

6.2. A megoldástér pontjainak osztályozása



12. ábra. Megoldástér elágazás-korlátozás bejárás során.

Ugyanaz a korlátozás módszere alkalmazható, mint amit a visszalépéses keresés során alkalmaztunk a Lehet() művelettel.
Ha prioritási sort használunk adagolónak és célfüggvény szerinti minimális megoldást keresünk, akkor további korlátozásokat alkalmazhatunk.
Tegyük fel, hogy minimalizációs feladatot kell megoldani. Tehát adott a `C(X)` valós értékű célfüggvény, és olyan `X` megoldást keresünk, amelyre a célfüggvény `C(X)` értéke minimális.

Tegyük fel továbbá, hogy a megoldáskezdeményekre meg tudunk adni olyan `AK(X)` *alsó korlát* függvényt, amelyekre teljesülnek az alábbi egyenlőtlenségek.
Bármely `X` megoldáskezdeményre és minden olyan `Y` megoldásra, amely leszármazottja `X`-nek:

$$\texttt{AK(X)} \leq \texttt{Cf(Y)}$$

Ekkor az adagoló lehet az `AK` szerinti minimumos prioritási sor, tehát az aktív pontok közül mindig a legkisebb alsó korlátú pontot választjuk aktuálisnak.
Az keresés során tároljuk az addig megtalált megoldások célfüggvény értékeinek minimumát.

$$\texttt{G_F_K} = \texttt{min\{Cf(X) : Megoldas(X)} \text{ és } \texttt{X} \text{ bevégzett}\texttt{\}}$$

Ekkor a következő kizárásokkal (korlátozásokkal) gyorsíthatjuk a keresést:

1. Ha az `X` új aktuális pontra `G_F_K < AK(X)`, akkor `X` kizárt lehet.
2. Ha a sorból az `X` pontot vettük ki és `G_F_K < AK(X)`, akkor a keresést befejezhetjük, hiszen nem kaphatunk már jobb megoldást.

6.3. Megoldás keresése alsó korlát szerinti minimumos prioritási sorral

```
FaPont BBKeres1(FaPont X){
  int G_F_K=INT_MAX;
  priority_queue<FaPont, vector<FaPont>,Rendezo> S;
  FaPont M;
  if (!X.Lehet()) return M;
  S.push(X);
  while (S.size()>0 ){
    X=FaPont(S.top()); S.pop();
    if (G_F_K<=X.AK()) return M;
    if(!X.Efiu()) continue;
    do{
      if (X.Lehet() && X.AK()<G_F_K ){
        if (X.Megoldas() && X.Cf()<G_F_K){//X jobb megoldás
          G_F_K=X.Cf();
          M=X;
        }
        S.push(X);
      }//else X kizárt lesz
    }while(X.Testver());
  }//while
  return M;
}
```

6.4. Megoldás keresése felső korlát szerinti minimumos prioritási sorral

Ha a megoldáskezdeményekre meg tudunk adni felső korlát is, akkor az adagoló lehet a felső korlát szerinti minimumos prioritási sor. Felső korlát olyan `FK(X)` függvényt értünk, amelyre teljesül, hogy minden `X` megoldáskezdeményre és minden olyan `Y` megoldásra, amely leszármazottja `X`-nek:

$$\texttt{Cf(Y)} \leq \texttt{FK(X)}$$

Ekkor azonban a 2. kizárást nem alkalmazhatjuk.
Az `FK(X)` felső korlátot erős felső korlátnak nevezzük, ha bármely `X` megoldáskezdeményre:

$$\texttt{FK(X)} < \infty \Rightarrow (\exists \texttt{Y})(\texttt{Y} \trianglelefteq \texttt{X} \wedge \texttt{Megoldas(Y)} \wedge \texttt{Cf(Y)} \leq \texttt{FK(X)})$$

Az erős felső korlát létezése azt jelenti, hogy bármely `X` megoldáskezdeményre, ha `FK(X)` $< \infty$, akkor biztosan létezik megoldás az `X` gyökerű megoldástér fában, és ennek célfüggvény értéke $\leq$ `FK(X)`. Ekkor a keresést végezhetjük a felső korlát szerinti minimumos prioritási sorral úgy, hogy feljegyezzük az addig érintett pontok felső korlátjainak minimumát. Legyen ez a `G_F_K`. Tehát a keresés során azt állíthatjuk, hogy biztosan létezik olyan megoldás, amelynek célfüggvény értéke $\leq$ `G_F_K`, de nem biztos, hogy már találtunk is ilyen megoldást. A 2. számú kizárást ennek ellenére továbbra is megtehetjük.

```
1   FaPont BBKeres2(FaPont X){
2     int G_F_K=INT_MAX;
3     priority_queue<FaPont, vector<FaPont>,Rendezo> S;
4     FaPont M;//az optimális megoldás
5     if (!X.Lehet()) return M;
6     S.push(X);
7     while (S.size()>0 ){
8       X=FaPont(S.top()); S.pop();
9       if (G_F_K<X.AK()) continue;//X kizárt lett
10      if(!X.Efiu()) continue; //X kizárt lett
11      do{
12        if (X.Lehet() && X.AK()<G_F_K ){
13          if (X.Megoldas() && X.Cf()<=G_F_K){
14            //X jobb megoldás
15            G_F_K=X.Cf();
16            M=X;
17          }else if(X.FK()<G_F_K)
18            G_F_K=X.FK();
19          S.push(X);
20        }//else X kizárt lesz
21      }while(X.Testver());
22    }//while
23    return M;
24  }
```

6.5. Optimális pénzváltás

Az elágazás-korlátozás módszer alkalmazása az optimális pénzváltás probléma megoldására.
Probléma: Optimális pénzváltás
Bemenet: $P = \{p_1, \ldots, p_n\}$ pozitív egészek halmaza, és K pozitív egész szám, a kifizetendő összeg.
Kimenet: Olyan $S \subseteq P$, hogy $\sum_{p \in S} = K$ és $|S| \to$ minimális.
Tegyük fel, hogy a pénzek nagyság szerint nemcsökkenő sorrendbe rendezettek: $p_1 \geq \ldots \geq p_n$. A megoldást keressük $X = \langle i_1, \ldots, i_m \rangle$, $i_1 < i_2 < \ldots < i_m$ alakú vektor formában. jelölje $Resz = \sum_{k=1}^{m} p_{i_k}$ és $Maradt = \sum_{j=i_m+1}^{n} p_j$ összegeket.

$$AK(X) = m + \lceil (K - Resz)/p_{i_m+1} \rceil$$

$$FK(X) = m + \lceil (K - Resz)/p_n \rceil$$

Ez a felső korlát nem erős, és nem is tudunk olcsón kiszámítható erős felső korlátot adni.

```
1  //Az optimális pénzváltás algoritmusa
2  #include<bits/stdc++.h>
3  using namespace std;
4  vector<int> P;//a pénzek
5  int K; //a kifizetendő érték
6
7  class FaPont{
8    //Globális: K, P
9    //probléma-specifikus adattagok
10   int resz=0; int maradt=0;
11   vector<int> A; //megoldáskezdemény
12   public:
13   bool Elso();
14   bool Testver();
15   bool Lehet();
16   bool Megoldas();
17   int AK();//az alsó korlát függvény
18   //
19   FaPont(){for(int x:P) maradt+=x;};
20   int Cf(){return A.size();}//a célfüggvény
21   void KiIr(){
22     if(A.size()==0)
23     cout<<"Nincs megoldás\n";
24     else{
25       cout<<K<<"="; for(int x:A) cout<<P[x]<<"+"; cout<<endl;
26     }
27   }
28 };
```

```
29 int FaPont:: AK(){
30   if(A.size()==0)
31   return (K+P[P.size()-1]-1)/P[P.size()-1];
32   if(A[A.size()-1]==P.size()-1){return A.size();};
33   return A.size()+(K-resz+P[A[A.size()-1]+1]-1)/P[A[A.size()-1]+1]
34 }
35 struct Rendezo{//a prioritási sor rendezési relációja
36   Rendezo(){};
37   bool operator() (FaPont& X, FaPont& Y) const{
38     return X.AK()>Y.AK();
39   }
40 };
41 FaPont BBKeres(FaPont X){
42   int G_F_K=INT_MAX;
43   priority_queue<FaPont, vector<FaPont>, Rendezo> S;
44   FaPont M;
45   if (!X.Lehet()) return M;
46   S.push(X);
47   while (S.size()>0 ){
48     X=FaPont(S.top()); S.pop();
49     if (G_F_K<=X.AK()) return M;
50     if(!X.Elso()) continue;
51     do{
52       if (X.Lehet() && X.AK()<G_F_K ){
53         if (X.Megoldas() && X.Cf()<G_F_K){//X jobb megoldás
54           G_F_K=X.Cf();
55           M=X;
56         }
57         S.push(X);
58       }//else X kizárt lesz
59     }while(X.Testver());
60   }//while
61   return M;
62 }
```

```
149 bool FaPont::Lehet(){
150   return resz<=K && K<=resz+maradt;
151 }
152 bool FaPont::Elso(){
153   if(A.size()==0){
154     A.push_back(0);
155     resz=P[0];
156     maradt-=P[0];
157     return true;
158   }
159   int x=A[A.size()-1];
160   if(x<P.size()-1){
161     A.push_back(x+1);
162     resz+=P[x+1];
163     maradt-=P[x+1];
164     return true;
165   }else return false;
166 }
167 bool FaPont::Testver(){
168   if(A.size()==0) return false;
169   int x=A[A.size()-1];
170   if(x<P.size()-1){
171     resz-=P[x]; resz+=P[x+1];
172     maradt-=P[x+1];
173     A[A.size()-1]=x+1;
174     return true;
175   }else{
176     return false;
177   }
178 }
179 bool FaPont::Megoldas(){return K==resz;}
180
181 int main(){
182   P={9,6,3,2,1}; //a pénzek
183   K=12; //a kifizetendő érték
184   FaPont X=FaPont();//X a megoldástér gyökere
185   X=BBKeres(X);
186   X.KiIr();
187 }
```

6.6. Ütemezési probléma

Bemenet:
$M = \{m_1, \ldots, m_n\}$ munkák halmaza
$m[i].idotartam > 0$ egész
$m[i].hatarido > 0$ egész
$m[i].haszon > 0$ egész
Kimenet:
$H \subseteq 1..n$
1. A H-beli munkák beoszthatók határidőt nem sértő módon.
2.

$$\overline{Cf}(H) = \sum_{i \in H} m[i].haszon \to maxi \tag{1}$$

H elemeinek egy $\langle i_1, \ldots, i_k \rangle$ felsorolása határidőt nem sértő, ha $\forall 1 \leq j \leq k$

$$\sum_{u=1}^{j} m[i_u].idotartam \leq m[i_j].hatarido \tag{2}$$

Állítás: H-nak akkor és csak akkor van határidőt nem sértő beosztása, ha elemeinek határidő szerinti felsorolása határidőt nem sértő.
$\Leftarrow$ triviális.
$\Rightarrow$ Tfh. H-nak van határidőt nem sértő beosztása, de ebben van olyan egymást követő u és $u+1$, hogy $m[i_u].hatarido > m[i_{u+1}].hatarido$. Ekkor u és $u+1$ felcserélhető a sorban.
Visszavezetés minimalizációs feladatra.

$$\begin{aligned} Cf(H) &= \sum_{i \notin H} m[i].haszon \\ &= \sum_{i=1}^{n} m[i].haszon - \overline{Cf}(H) \to mini \end{aligned}$$

$\overline{Cf}(H) \to maxi \Leftrightarrow Cf(H) \to mini$
Tegyük fel, hogy a munkák határidő szerint nemcsökkenő sorrendben vannak felsorolva. Ekkor a megoldás kifejezhető $X = \langle i_1, \ldots, i_k \rangle$ vektorral, ahol $i_1 < i_2 < \cdots < i_k$
Minden X megoldáskezdeményre definiáljuk. a problémaspecifikus műveleteket.
$Lehet(X) = igaz \Leftrightarrow$ ha a felsorolás határidőt nem sértő.
$Megoldas(X) = Lehet(X)$

$$AK(X) = \sum_{j < i_k,\, j \notin X} m[j].haszon \tag{3}$$

$$FK(X) = \sum_{j \notin X} m[j].haszon \tag{4}$$

Tehát, ha $Lehet(X)$, akkor $Megoldas(X)$ és $Cf(X) = FK(X)$, ezért FK erős felső korlát.

```
1  #include<bits/stdc++.h>
2
3  using namespace std;
4  vector<int> Hatarido;
5  vector<int> Haszon;
6  vector<int> Vido;
7
8  class FaPont{
9    //Globalis: Hatarido, Haszon, Vido
10   int oido=0;      //a beválasztott munkák összideje
11   int ehaszon=0;  //az elmaradt haszon
12   int maradt;      //a még választható munkák haszna
13   vector<int> A;  //megoldáskezdemény
14   public:
15   bool Elso();
16   bool Testver();
17
18   FaPont(){//konstruktor
19     oido=0; ehaszon=0;  maradt=0;
20     for(int x:Haszon) maradt+=x;
21   }
22   //
23   int Cf(){return ehaszon+maradt;}//a célfüggvény
24   int AK(){return ehaszon;}
25   int FK(){return ehaszon+maradt;}
26   bool Lehet(){
27     return A.size()==0 || oido<=Hatarido[A[A.size()-1]];
28   }
29   bool Megoldas(){
30     return A.size()==0 || oido<=Hatarido[A[A.size()-1]];
31   }
32   void KiIr(){
33     if(A.size()==0){cout<<"Mincs megoldás"; return;}
34     for(int x:A) cout<<x+1<<","; cout<<endl;
35   }
36 };
```

```
189 struct Rendezo{//a prioritási sor rendezési relációja
190   Rendezo(){};
191   bool operator() (FaPont& X, FaPont& Y) const{
192     return X.FK()>Y.FK();
193   }
194 };
195 //Megoldás keresés felső korlát szerinti minimumos prioritási sorr
196 FaPont BBKeres2(FaPont X){
197   int G_F_K=INT_MAX;
198   priority_queue<FaPont, vector<FaPont>,Rendezo> S;
199   FaPont M;//az optimális megoldás
200   if (!X.Lehet()) return M;
201   S.push(X);
202   while (S.size()>0 ){
203     X=FaPont(S.top()); S.pop();
204     if (G_F_K<X.AK()) continue; //X kizárt lett
205     if(!X.Elso()) continue;    //X kizárt lett
206     do{
207       if (X.Lehet() && X.AK()<G_F_K ){
208         if (X.Megoldas() && X.Cf()<=G_F_K){//X jobb megoldás
209           G_F_K=X.Cf();
210           M=X;
211         }else if(X.FK()<G_F_K)
212           G_F_K=X.FK();
213         S.push(X);
214       }//else X kizárt lesz
215     }while(X.Testver());
216   }//while
217   return M;
218 }
```

```
bool FaPont::Elso(){
  if(A.size()==0){
    A.push_back(0);
    maradt-=Haszon[0];
    oido+=Vido[0];
    return true;
  }
  int x=A[A.size()-1];
  if(x<Haszon.size()-1){
    A.push_back(x+1);
    maradt-=Haszon[x+1];
    oido+=Vido[x+1];
    return true;
  }else
  return false;
}
bool FaPont::Testver(){
  if(A.size()==0) return false;
  int x=A[A.size()-1];
  if(x<Haszon.size()-1){
    ehaszon+=Haszon[x];
    oido-=Vido[x]; oido+=Vido[x+1];
    maradt-=Haszon[x+1];
    A[A.size()-1]++;
    return true;
  }else{
    return false;
  }
}

int main(){
  Hatarido={1,2,2,6,9};//határidő
  Vido={1,6,9,2,1};//vidő
  Haszon={9,6,3,2,7};//haszon
  FaPont X=FaPont();//X a megoldástér gyökere
  FaPont M=BBKeres2(X);
 M.KiIr();
}
```

7. Feladat 1: Mester:Rekurzív adatszerkezetek: Tárgyak sorban.

Sorban egymás után N tárgy helyezkedik el. Egy robot megy végig a soron és minden tárgyra meg kell mondania, hogy előtte a sorban hány olyan tárgy van, amely az adott tárgynál nem magasabb.
Készíts programot, amely megoldja a robot feladatát!

Bemenet

Bemenet A standard bemenet első sorában van a tárgyak `N` száma. A következő `N` sor mindegyike egy tárgy m_i magassági értékét tartalmazza.

Kimenet

Kimenet A standard kimenet első és egyetlen sorába `N` számot kell írni, az i-edik szám azon tárgyak száma legyen, amelyek megelőzik a sorban az i-edik tárgyat és magasságuk legfeljebb akkora, mint az i-edik tárgyé!

Példa

Bemenet	Kimenet
7 9 19 26 28 4 28 71	0 1 2 3 0 5 6

Korlátok

Korlátok $1 < N \leq 100\,000$, $1 \leq m_i \leq 100\,000\,000$
Időlimit: 0.1 másodperc
Memórialimit: 64 MB.

Pontozás

Pontok A pontok 10%-át lehet szerezni olyan bemenetekre, ahol $N \leq 1000$.
A pontok további 40%-át lehet szerezni olyan bemenetekre, ahol $N \leq 10\,000$
A pontok további 50%-át lehet szerezni olyan bemenetekre, ahol nincs egyéb korlátozás.

8. Feladat 2: Mester: Visszalépéses algoritmusok: Közbeszerzés.

Valahol országban egy nagyszabású állami beruházás megvalósításán dolgoznak. A beruházás terve `M` részfeladat elvégzését írja elő. Közbeszerzési pályázatot írtak ki, amelyre `V` vállalkozó jelentkezett, mindegyik megadta, hogy mely munkát tudja elvégezni, és azt milyen költséggel. Ebben az országban nagy gondot fordítanak a korrupció elkerülésére, ezért olyan vállalkozókat akarnak kiválasztani a pályázók közül, hogy az összköltség a lehető legkisebb legyen. Mivel a munkákat párhuzamosan kell elvégezni, ezért minden pályázó csak egy feladatra kaphat megbízást. Készíts programot, amely kiszámítja, hogy az egyes feladatok elvégzésére melyik pályázót bízzák meg!

Bemenet

Bemenet A standard bemenet első sorában van a feladatok `M` száma és a pályázók `V` száma. A fel-adatokat 1-től `M`-ig, a vállalkozókat 1-től `V`-ig sorszámukkal azonosítjuk. A következő `M` sor mindegyike `V` darab egész számot tartalmaz. Az i-edik sorban az j-edik szám azt adja meg, hogy az i-edik feladatot a j-edik pályázó mekkora költséggel vállalná el. Ha ez a szám 0, az azt jelenti, hogy a vállalkozó ezt a feladatot nem vállalja el.

Kimenet

Kimenet A standard kimenet első és egyetlen sorába a lehető legkisebb összköltséget kell írni. A második sor M egész számot tartalmazzon, az i-edik szám annak a vállalkozónak a sorszáma le-gyen, aki az i-edik feladatot fogja elvégezni! Ha nincs a feltételeknek megfelelő megoldás, akkor az első és egyetlen sorba a 0 számot kell kiírni! Több megoldás esetén bármelyik megadható.

Példa

Bemenet	Kimenet
4 7 2 3 0 7 2 3 0 0 2 0 0 8 1 0 9 1 3 0 0 7 5 3 1 1 0 0 8 8	5 5 6 2 3

Korlátok

Korlátok $1 < M \leq 16$, $1 \leq M \leq V \leq 20$
Időlimit: 2.0 másodperc
Memórialimit: 512 MB.

Pontozás

Pontok A pontok 20%-át lehet szerezni olyan bemenetekre, ahol $V \leq 10$.
A pontok további 30%-át lehet szerezni olyan bemenetekre, ahol $V \leq 15$.
A pontok további 50%-át lehet szerezni olyan bemenetekre, ahol nincs egyéb korlátozás.

9. Gyakorló feladatok

Mester/Visszalépéses keresés: 4. Megye
Mester/Visszalépéses keresés: 22. Munkásfelvétel
Mester/Visszalépéses keresés: 18. L üzlet K pékség